The Learning Rate (α)

The **learning rate**, denoted by the Greek letter α (alpha), is a critical parameter in the gradient descent algorithm. It controls the size of the steps taken on each iteration, and choosing an appropriate value is essential for the algorithm's performance.

If the Learning Rate is Too Small 🐢

- **Behavior:** The algorithm takes extremely small "baby steps" on each iteration.
- **Consequence:** Gradient descent will work, but it will be very **slow to converge**. It may take an excessive number of iterations to reach the minimum.

If the Learning Rate is Too Large 🚀

- **Behavior:** The algorithm takes huge steps, which can cause it to "overshoot" the minimum of the cost function. With each step, it might land on a point with an even higher cost than the previous one.
- **Consequence:** Gradient descent may **fail to converge** or even **diverge**, meaning the cost gets worse over time, moving further and further away from the minimum.

Behavior at a Local Minimum ✅

- When the algorithm reaches a local minimum, the slope of the tangent line to the cost function is **zero**.
- This means the **derivative term in the update rule is zero**.
- The update becomes: $w := w - \alpha \cdot 0$, which simplifies to $w := w$.
- **Consequence:** The parameter `w` stops changing. The algorithm has successfully **converged** and will remain at the local minimum.

Natural Decrease in Step Size

- Even with a **fixed learning rate** α, the step sizes taken by gradient descent will **automatically get smaller** as the algorithm approaches a minimum.
- **Why?** As you get closer to the bottom of the "bowl," the slope of the cost function becomes less steep. This means the derivative term gets smaller, resulting in a smaller update to the parameters.
- This property allows the algorithm to take large steps when it's far from the minimum and smaller, more precise steps as it gets closer, helping it converge smoothly.